1. Objetivo
2. Ambiente experimental
3. Projeto experimental (DOE)
4. Dados e validação
5. Precisão de $\hat{\pi}$
6. Desempenho
7. Dificuldades encontradas e soluções
8. Limitações e próximos passos
9. Uso de ferramentas de IA (requisito 5.3)
Apêndice: sessão R

Code

Estimativa de π por Monte Carlo com OpenMP — Análise parcial (Entrega 2)

Lucas Machado

1. Objetivo

Este documento é o caderno de análise (programação literal) da Entrega 2: ele lê os CSVs produzidos pelo pipeline (`make run`), valida os dados e analisa (i) a **precisão** de $\hat{\pi}$ em função de N e (ii) o **desempenho** (tempo, speedup, eficiência) em função de N e do número de threads, conforme as métricas definidas na Entrega 1. Tudo que aparece em número no texto é calculado a partir dos dados; reexecutar `make relatorio` regenera o documento inteiro.

2. Ambiente experimental

O ambiente é lido de `dados/ambiente.txt` , registrado **no momento da medição** por `scripts/registra_ambiente.sh` (e não no momento em que este relatório é gerado, que pode ser em outra máquina).

Show

Item	Valor
DATA	2026-11-09T07:56:59-03:00
CPU_MODELO	AMD Ryzen 5 5500U with Radeon Graphics
CPUS_LOGICAS	12
NUCLEOS_FISICOS	6

Item	Valor
SO	Fedora Linux 43 (Workstation Edition)
KERNEL	7.2.13-100.fc43.x86_64
COMPILADOR	g++ (GCC) 16.2.0
FLAGS	-fopenmp -O3
GOVERNOR	powersave
LOADAVG_INICIO	4.28 2.67 2.39
OMP_ENV	
FLAKE_LOCK_SHA256	6132d7d2a7391542
GIT_COMMIT	c5415a9
GIT_SUJO	2
LOADAVG_FIM	2.59 2.97 2.69

Show

Detalhes de medição: o tempo (`omp_get_wtime`) cobre a região paralela inteira (criação das threads, geração dos números e classificação dos pontos). Antes da bateria é feita uma execução de aquecimento descartada. **O "tempo sequencial" T(1) usado no speedup é o binário OpenMP com 1 thread**, não uma versão compilada sem OpenMP (ver Limitações).

3. Projeto experimental (DOE)

Show

Repetições por célula do fatorial N × Threads

N	p=1	p=2	p=4	p=8
1e+04	5	5	5	5
1e+06	5	5	5	5
1e+08	5	5	5	5
1e+09	5	5	5	5

Fatorial completo com 4 valores de N × 4 valores de threads × 5 repetições = **80 execuções**, geradas por `scripts/gera_layout.R` com semente fixa. A **ordem de execução é aleatória** (coluna `Ordem`), de modo que qualquer deriva do sistema (temperatura, processos concorrentes) não se confunda com um fator. Cada execução tem uma **semente própria e única** (coluna `Semente`), repassada ao C++, que deriva um fluxo independente por thread via `std::seed_seq` .

4. Dados e validação

Show

Verificacao	OK

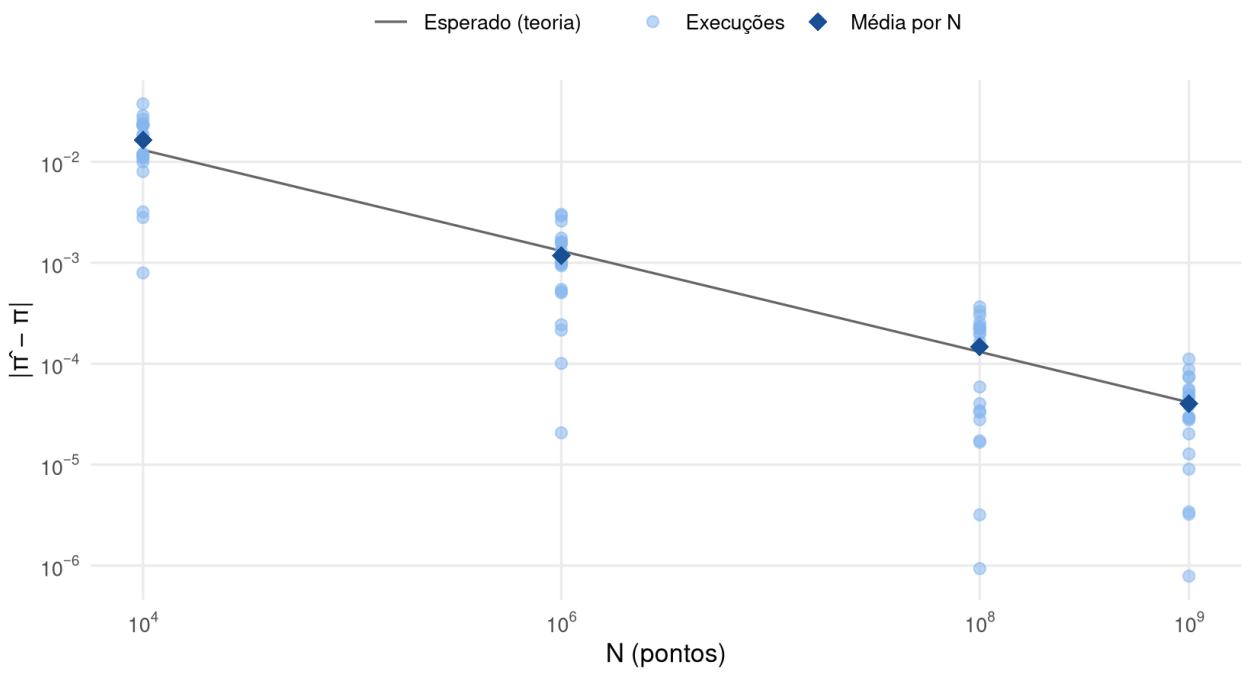
	Verificacao	OK
Nº de execuções = plano	Nº de execuções = plano	✔
Sem valores ausentes	Sem valores ausentes	✔
0 ≤ M ≤ N	0 ≤ M ≤ N	✔
π̂ = 4M/N (consistência)	π̂ = 4M/N (consistência)	✔
Execuções batem com o plano (N, p, rep, semente)	Execuções batem com o plano (N, p, rep, semente)	✔
Threads reais = threads pedidas	Threads reais = threads pedidas	✔
Sementes únicas	Sementes únicas	✔

Show

5. Precisão de π̂

Como M ~ Binomial(N, π/4), o desvio-padrão teórico de π̂ é σ(N) = √(π(4−π)/N) ≈ 1,64/√N, e o erro absoluto médio esperado é σ·√(2/π). Logo, o erro deve cair com **N^−1/2**: cada casa decimal a mais custa 100× mais pontos.

Show



Show

Erro padronizado z = (π̂−π)/σ(N), esperado: média≈0, dp≈1

Threads	n	media_z	dp_z	|z|>2
1	20	-0.23	1.10	1
2	20	-0.23	0.84	1
4	20	0.17	1.03	0

Threads	n	media_z	dp_z	|z|>2
8	20	0.01	1.21	2

A inclinação ajustada de log|erro| × log N é **-0.539** (IC95% [-0.604; -0.474]); o valor teórico é –0,5 (consistente com a teoria). O z padronizado tem média -0.07 e desvio-padrão 1.05 (esperado 0 e 1), o que valida gerador e estimador, e não depende do número de threads. **Conclusão parcial:** a paralelização não altera a precisão; ela só reduz o tempo para atingir um mesmo N.

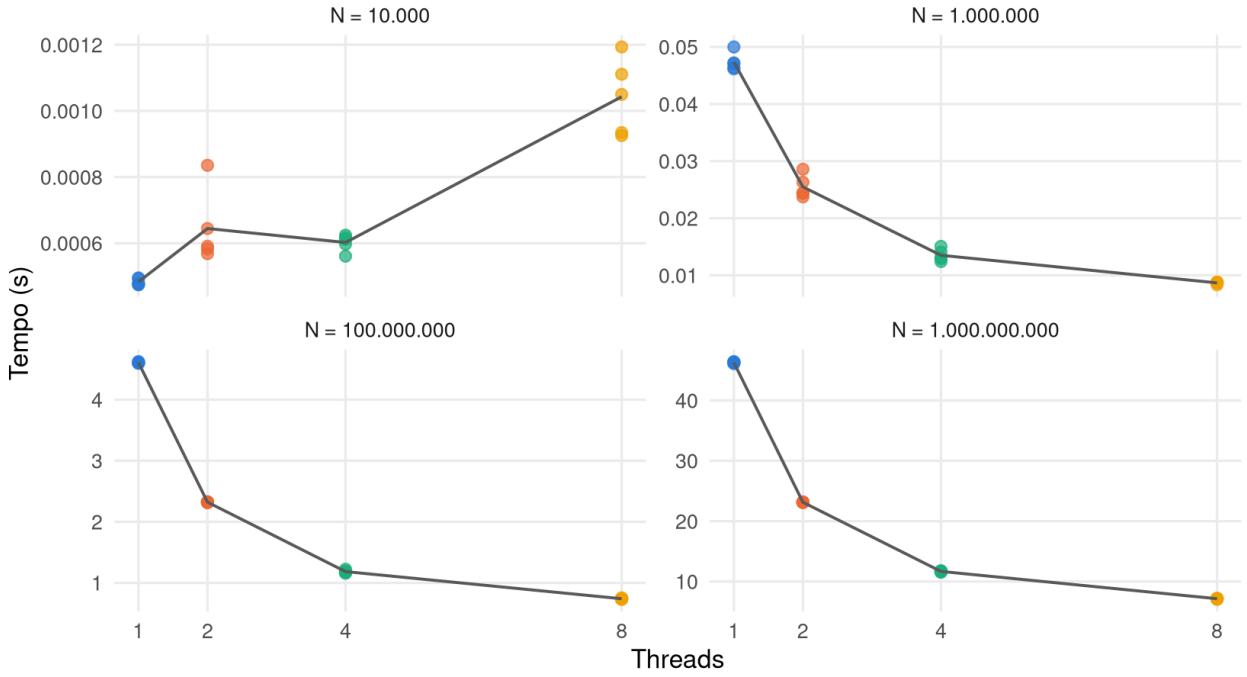
6. Desempenho

6.1 Tempo de execução

Show

N	Threads	n	Média (s)	Mediana (s)	DP (s)	CV (%)
1e+04	1	5	0.00048	0.00048	0.00001	2.1
1e+04	2	5	0.00064	0.00059	0.00011	17.2
1e+04	4	5	0.00060	0.00061	0.00003	4.2
1e+04	8	5	0.00104	0.00105	0.00011	11.0
1e+06	1	5	0.04737	0.04711	0.00154	3.2
1e+06	2	5	0.02548	0.02441	0.00198	7.8
1e+06	4	5	0.01350	0.01306	0.00105	7.8
1e+06	8	5	0.00867	0.00874	0.00019	2.2
1e+08	1	5	4.60788	4.60879	0.01039	0.2
1e+08	2	5	2.32177	2.32256	0.00582	0.3
1e+08	4	5	1.18475	1.18774	0.02533	2.1
1e+08	8	5	0.74015	0.73993	0.00940	1.3
1e+09	1	5	46.28183	46.29837	0.12118	0.3
1e+09	2	5	23.13726	23.12203	0.03333	0.1
1e+09	4	5	11.67786	11.73580	0.09589	0.8
1e+09	8	5	7.15238	7.16947	0.06981	1.0

Show



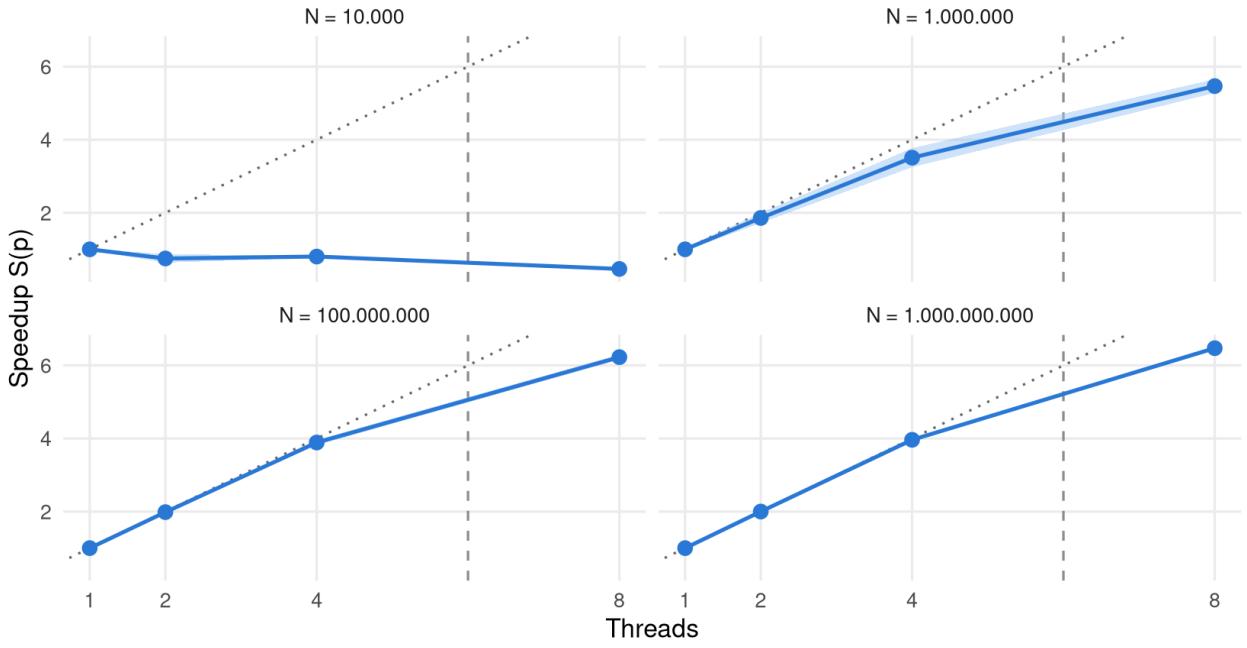
6.2 Speedup e eficiência

S(p) = T(1)/T(p) e E(p) = S(p)/p, sobre as médias por (N, p). O IC95% do speedup usa o método delta sobre os erros-padrão das médias.

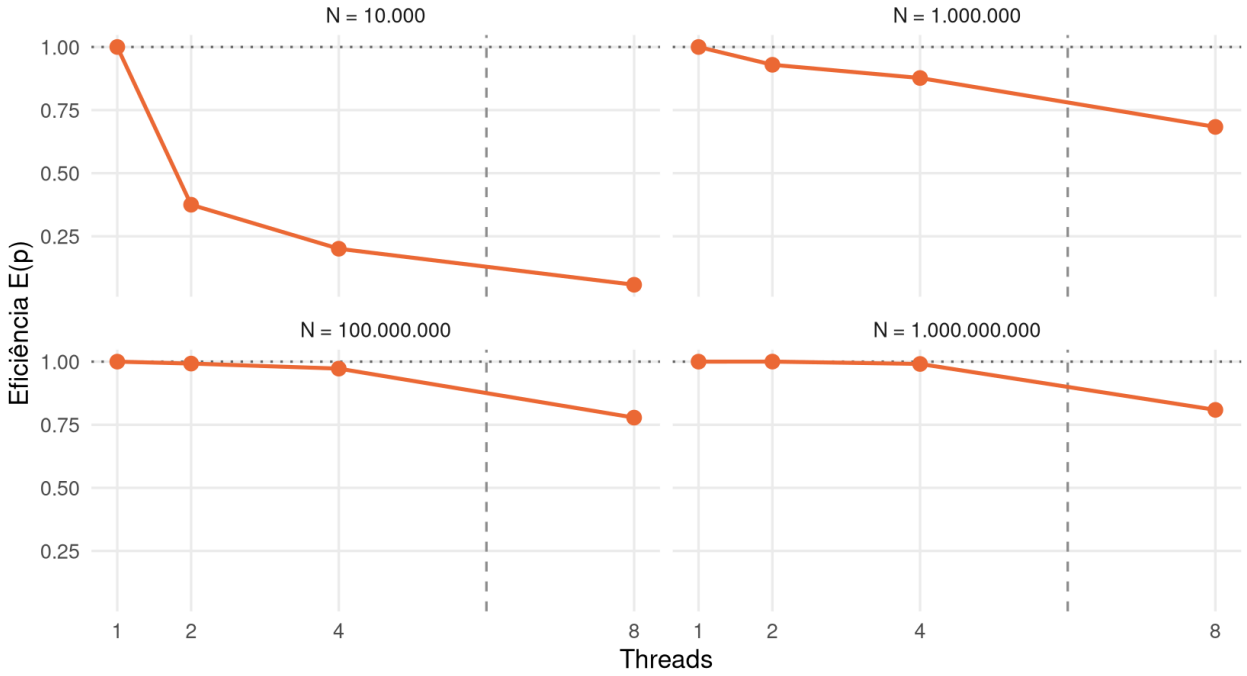
Show

N	Threads	Speedup	Eficiência	Mpontos/s
1e+04	1	1.00	1.00	20.7
1e+04	2	0.75	0.38	15.5
1e+04	4	0.80	0.20	16.6
1e+04	8	0.46	0.06	9.6
1e+06	1	1.00	1.00	21.1
1e+06	2	1.86	0.93	39.2
1e+06	4	3.51	0.88	74.1
1e+06	8	5.47	0.68	115.4
1e+08	1	1.00	1.00	21.7
1e+08	2	1.98	0.99	43.1
1e+08	4	3.89	0.97	84.4
1e+08	8	6.23	0.78	135.1
1e+09	1	1.00	1.00	21.6
1e+09	2	2.00	1.00	43.2
1e+09	4	3.96	0.99	85.6
1e+09	8	6.47	0.81	139.8

Show



Pontilhado: ideal S=p. Tracejado vertical: nº de núcleos físicos.



Show

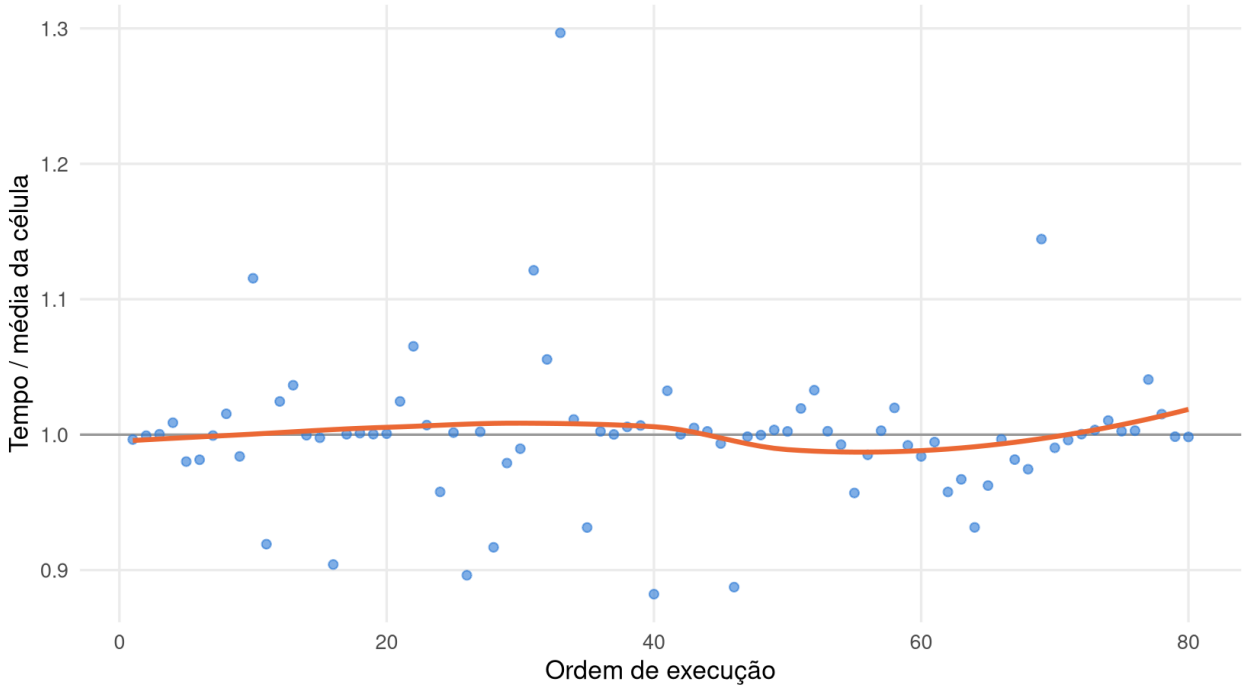
Leitura (números gerados dos dados). Para N = 1.000.000.000 e 8 threads, o speedup é 6.47 (eficiência 81%). Para N = 10.000, o speedup com 8 threads é 0.46. A máquina tem 6 núcleos físicos e 12 CPUs lógicas.

✎ **Interpretação do grupo (preencher/validar).** Expectativa a priori: o laço é *embaraçosamente paralelo* e o estado do `mt19937` (~2,5 KB/thread) cabe no cache L1, então espera-se speedup próximo do ideal **até o nº de núcleos físicos** e saturação depois (SMT/oversubscription). Para N pequeno (10^4), o custo de criar threads domina o trabalho útil e o speedup tende a ficar abaixo de 1. Hipóteses a confirmar com os gráficos acima: (a) queda de eficiência além dos núcleos físicos; (b) redução de frequência com mais núcleos ativos; (c) overhead fixo em N pequeno.

6.3 Controle experimental: deriva temporal

Como a ordem é aleatória, uma deriva do sistema aparece como ruído, não como viés. Verificação: tempo de cada execução normalizado pela média da sua célula (N, p) em função da ordem.

Show



Correlação de Spearman entre ordem e tempo relativo: **-0.02** (sem deriva relevante).

7. Dificuldades encontradas e soluções

Dificuldade (identificada na auditoria do pipeline)	Solução
Sementes `12345 + thread + repeticao` **repetem fluxos** entre repetições (ex.: thread 1 da rep. 1 = thread 0 da rep. 2, mesmo M); repetições não eram independentes e o desvio-padrão ficava subestimado	Semente única por execução (gerada no DOE), `seed_seq` por thread; teste `make test`
Execução em ordem crescente de N/threads: deriva do sistema se confunde com os fatores	Ordem aleatória no DOE + coluna `Ordem` + verificação de deriva (§6.3)

Dificuldade (identificada na auditoria do pipeline)	Solução
`tail | while … exit 1` só saía do subshell: falha de execução ainda imprimia "sucesso"	Laço sem pipe, checagem de campos, saída escrita em `.tmp` e promovida só no final
`make clean` apagava `dados/` (medições) e `make relatorio` falhava sem regra para o CSV	`clean` preserva `dados/`; `distclean` apaga tudo; CSV só é gerado se não existir
Ambiente não registrado / semente não registrada / saída com 6 dígitos	`registra_ambiente.sh` na hora da medição; `Semente` e `Threads_Reais` no CSV; `setprecision(10)`

8. Limitações e próximos passos

- T(1) é OpenMP com 1 thread, e não uma versão sequencial pura como prometido na Entrega 1; compilar sem `-fopenmp` (com `#ifdef _OPENMP`) fecharia essa lacuna.
- Uma única máquina e um único gerador (`mt19937`); sem fixação de threads (`OMP_PROC_BIND`).
- Para a Etapa 3: modelo linear completo (log T ~ log N + log p), template ACM, `renv.lock`.

9. Uso de ferramentas de IA (requisito 5.3)

- **Agente:** Claude (Anthropic), modelo Sonnet 5.5, em modo agêntico (execução de comandos num sandbox).
- **Dados fornecidos:** `monte_carlo.cpp`, `executor.sh`, `gera_layout.R`, `Makefile`, e os PDFs da Entrega 1 e do enunciado.
- **Produzido com auxílio da IA:** correção de sementes no C++, `gera_layout.R`, `executor.sh`, `registra_ambiente.sh`, `Makefile`, este caderno e a tabela da seção 7.
- **Verificação humana:** *a preencher pelo grupo* (revisão do código, conferência dos gráficos e da interpretação da seção 6.2).

Apêndice: sessão R

Show

```
## R version 4.6.1 (2026-06-24)
## Platform: x86_64-pc-linux-gnu
## Running under: Fedora Linux 43 (Workstation Edition)
##
## Matrix products: default
## BLAS/LAPACK: /nix/store/rcqarsssrqbj4js5rxppiabc2d6hjfm0-blas-3/lib/libbla
s.so.3;  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C           LC_TIME=C.UTF-8
##  [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
##  [7] LC_PAPER=C.UTF-8       LC_NAME=C              LC_ADDRESS=C
## [10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C
##
## time zone: America/Sao_Paulo
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] scales_1.4.0  ggplot2_4.0.3 readr_2.2.0   tidyr_1.3.2   dplyr_1.2.1
##
## loaded via a namespace (and not attached):
##  [1] sass_0.4.10        generics_0.1.4     lattice_0.23-1     hms_1.1.4
##  [5] digest_0.6.39      magrittr_2.0.5     evaluate_1.0.5     grid_4.6.1
##  [9] RColorBrewer_1.1-3 fastmap_1.2.0      jsonlite_2.0.0     Matrix_1.7-6
## [13] mgcv_1.9-4         purrr_1.2.2        textshaping_1.0.5  jquerylib_0.
1.4
## [17] cli_3.6.6          rlang_1.3.0        crayon_1.5.3       bit64_4.8.6
## [21] splines_4.6.1      withr_3.0.3        cachem_1.1.0       yaml_2.3.12
## [25] otel_0.2.0         tools_4.6.1        parallel_4.6.1     tzdb_0.5.0
## [29] vctrs_0.7.3        R6_2.6.1           lifecycle_1.0.5    bit_4.6.0
## [33] vroom_1.7.1        ragg_1.5.2         pkgconfig_2.0.3    pillar_1.11.
1
## [37] bslib_0.12.0       gtable_0.3.6       glue_1.8.1         systemfonts_
1.3.2
## [41] xfun_0.61          tibble_3.3.1       tidyselect_1.2.1   knitr_1.52
## [45] farver_2.1.2       htmltools_0.5.9    nlme_3.1-171       rmarkdown_2.
32
## [49] labeling_0.4.3     compiler_4.6.1     S7_0.2.2
```